



KATHOLISCHE UNIVERSITÄT
EICHSTÄTT-INGOLSTADT

KATHOLISCHE UNIVERSITÄT EICHSTÄTT-INGOLSTADT

Faculty of Mathematics and Geography

Bachelor's Programme in Data Science

Universal Approximation, Attention Layers, Transformers, and Sumformers

Academic Seminar Report

Author: Ali Guliyev

Supervisor: Felix Voigtlaender

Date: June 2, 2026

Abstract

This report accompanies a seminar presentation delivered for the Data Science programme at the Catholic University of Eichstätt-Ingolstadt. It covers four interconnected topics in modern deep learning theory: the Universal Approximation Theorem (UAT) for single-hidden-layer neural networks, the definition and permutation-equivariant structure of attention layers, the architecture of Transformers and their continuity properties, and the recently introduced Sumformer—a simplified architecture that achieves universal approximation of continuous permutation-equivariant sequence-to-sequence functions. The presentation draws on classical results by Cybenko and Hornik as well as more recent work by Alberti et al. (2023) and Chen et al. (2025), providing both theoretical foundations and illustrative experiments.

Contents

1	Introduction	2
2	Universal Approximation Theorem	2
2.1	Statement and Assumptions	2
2.2	Conceptual Proof Sketch	2
2.3	Width versus Depth	3
3	Attention Layers	5
3.1	Definition	5
3.2	Permutation Equivariance	5
3.3	When to Break Equivariance	6
3.4	Computational Complexity	6
4	Transformers	6
4.1	Architecture	6
4.2	Permutation Equivariance of Transformers	7
5	Sumformers and Universal Approximation	7
5.1	Motivation	7
5.2	Definition and Theorem	7
5.3	Sumformers in Practice: Where They Fall Short	8
6	Conclusion	9

1 Introduction

A fundamental question in machine learning is: how expressive are the models we use? Can a given architecture approximate any target function, or are there inherent limitations? This question lies at the heart of the Universal Approximation Theorem for feedforward networks and, more recently, of theoretical analyses of Transformer models.

The seminar traces a path from a classical single-layer result to a contemporary sequence-to-sequence setting. Section 2 reviews the Universal Approximation Theorem, its assumptions, and an intuitive proof sketch. Section 3 introduces attention layers and proves their permutation-equivariant nature, together with a brief discussion of when equivariance is and is not desirable. Section 4 describes the Transformer block and explains why a Transformer without positional encodings implements a permutation-equivariant continuous function. Finally, Section 5 introduces Sumformers and the universal approximation theorem for sequence-to-sequence functions, together with an experiment showing where Sumformers fall short in practice compared to an encode-process-decode approach.

2 Universal Approximation Theorem

2.1 Statement and Assumptions

The classical Universal Approximation Theorem establishes that a single hidden layer is theoretically sufficient to approximate any continuous function to arbitrary precision.

Theorem 2.1 (Universal Approximation Theorem [Cybenko, 1989, Hornik et al., 1989]). Let f be any continuous function defined on a compact subset $K \subset \mathbb{R}^n$ and let $\varepsilon > 0$. Then there exists a single-hidden-layer neural network $G(\mathbf{x})$ of the form

$$G(\mathbf{x}) = \sum_{j=1}^N a_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j), \quad \mathbf{w}_j \in \mathbb{R}^n, \quad b_j, a_j \in \mathbb{R},$$

such that $|G(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$ for all $\mathbf{x} \in K$.

Four assumptions deserve attention. First, the target function must be continuous; truly discontinuous functions cannot be uniformly approximated by continuous networks, although in classification the network output itself remains continuous and the discontinuity arises only after thresholding. Second, the activation must be continuous and non-polynomial; common choices include the sigmoid $\sigma(x) = 1/(1 + e^{-x})$, hyperbolic tangent, and ReLU. Third, the theorem is an *existence* result: it guarantees that a sufficiently wide network exists, but gives no bound on how large N must be. Fourth, the error ε is measured *uniformly* over all $\mathbf{x} \in K$.

2.2 Conceptual Proof Sketch

A rigorous proof requires advanced functional-analysis machinery (e.g. the Hahn–Banach theorem or Stone–Weierstrass theorem), but the core intuition is accessible via a step-function argument [Nielsen, 2015]. Because f is continuous, it changes only slightly over sufficiently small regions. A sigmoid neuron with a large weight w approximates a smooth switch: for $w = 10$ and bias $b = -5$, the neuron activates sharply around $x = 0.5$. Two neurons with opposite-sign output weights create a bump that is nearly zero everywhere except on a small interval.

By stacking arbitrarily many neurons, the single hidden layer can tile the domain K with narrow bumps, each approximating a locally constant value of f . As illustrated

Theorem Let f be any continuous function defined on a compact subset $K \subset \mathbb{R}^n$ and let $\varepsilon > 0$.

Then there exists a single-hidden-layer neural network $G(\mathbf{x})$ of the form

$$G(\mathbf{x}) = \sum_{j=1}^N a_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j)$$

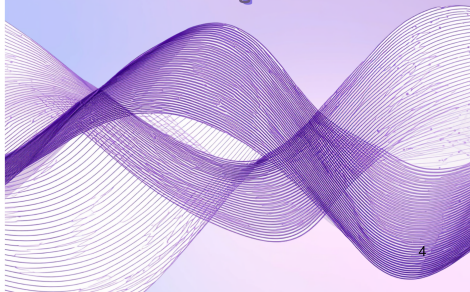
where $\mathbf{w}_j \in \mathbb{R}^n$, $b_j \in \mathbb{R}$, $a_j \in \mathbb{R}$ for $j = 1, \dots, N$, such that for all $\mathbf{x} \in K$,

$$|G(\mathbf{x}) - f(\mathbf{x})| < \varepsilon.$$

Key assumptions of UAT:

1. The target function f is continuous on a compact domain $K \subset \mathbb{R}^n$.
2. We assume that σ is continuous and non-polynomial, for example sigmoid, tanh, or ReLU.
3. The network G can be sufficiently wide.
4. The approximation error is measured uniformly on K .

UAT Theorem and Assumptions

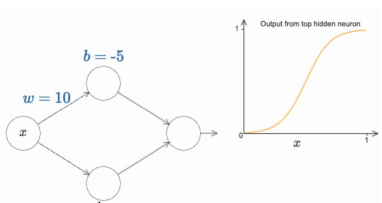


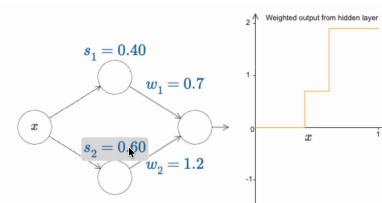
4

Figure 1: The UAT statement and its four key assumptions as presented in the seminar. The theorem requires (1) a continuous target f on a compact K , (2) a continuous non-polynomial activation σ (e.g. sigmoid, tanh, ReLU), (3) an unrestricted width N , and (4) uniform approximation on K .

Conceptual Proof

Rigorous proof of the Universal Approximation Theorem requires advanced mathematics. Since continuous functions change only slightly on sufficiently small regions, they can be approximated locally by nearly constant values. Sigmoid neurons act like smooth "switches":





$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

6

Source: Michael Nielsen's website: <http://neuralnetworksanddeeplearning.com/chap4.html>

Figure 2: Sigmoid neurons act as smooth switches (left). Combining two neurons with weights $w_1 = 0.7$ and $w_2 = 1.2$ and bias values $s_1 = 0.40$, $s_2 = 0.60$ produces a step-like output (right). Source: Nielsen [2015].

in Figure 3, four neurons already produce a recognisable step approximation, and the average deviation decreases as more neurons are added. The theoretical “average vertical deviation” corresponds to $\frac{1}{|K|} \int_K |f(x) - G(x)| dx$, which is related to but weaker than the uniform bound stated in the theorem.

2.3 Width versus Depth

The UAT guarantees universal approximation with a *single* hidden layer of unbounded width. A natural question is whether depth (more layers with moderate width) can replace width. Safran and Shamir [2020] provide evidence that depth often matters significantly

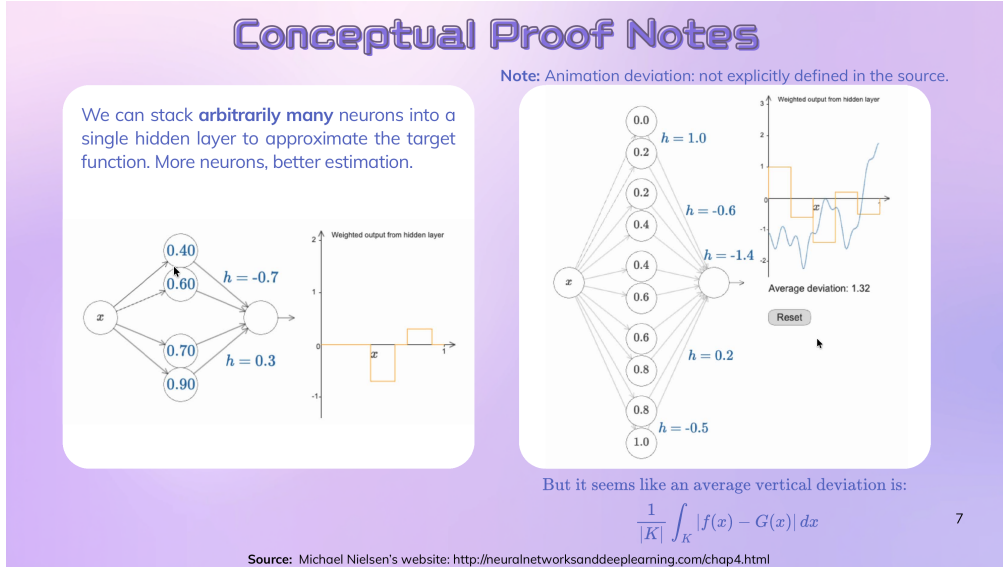


Figure 3: Stacking more neurons refines the bump-function approximation. Left: four neurons with three bumps. Right: ten neurons with multiple overlapping bumps and an average deviation of 1.32. Source: Nielsen [2015].

in practice. Figure 4 shows a controlled experiment: the task is to classify whether a point in $\mathbb{R}^{100}$ lies inside the unit L_2 -ball. A 3-layer network of width 100 (≈ 30 k parameters) outperforms 2-layer networks of width up to 800 (≈ 320 k parameters) on both training and validation RMSE, suggesting that depth can be exponentially more efficient than width for certain functions.

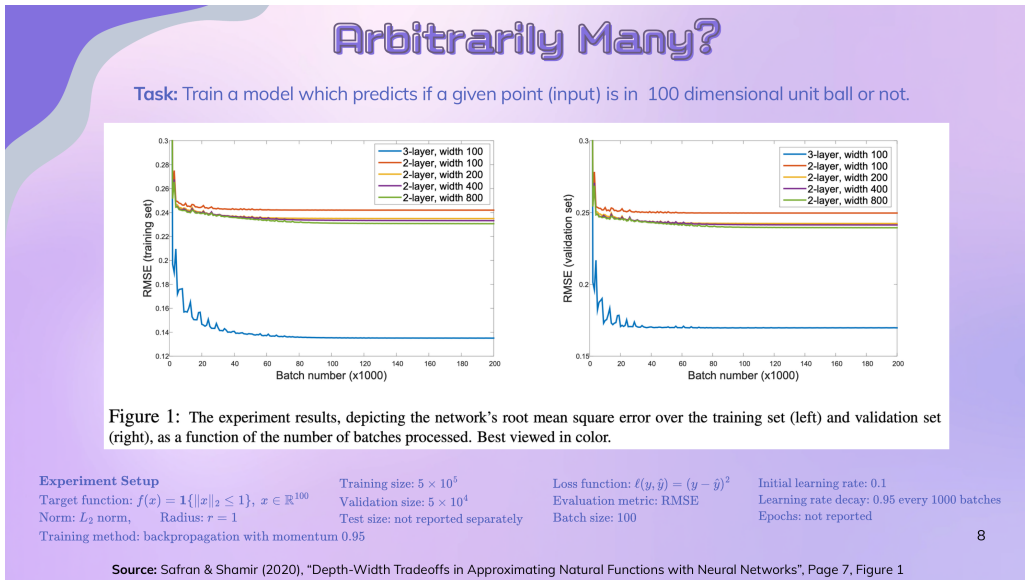


Figure 4: Training (left) and validation (right) RMSE for classifying membership in the 100-dimensional unit ball. A deeper 3-layer network (blue) consistently outperforms wider 2-layer networks, illustrating the practical relevance of depth beyond what the UAT alone implies. Source: Safran and Shamir [2020], Figure 1.

3 Attention Layers

3.1 Definition

Attention layers are the central building block of Transformers. Rather than processing each input token independently, an attention layer computes a context-dependent weighted combination of all tokens.

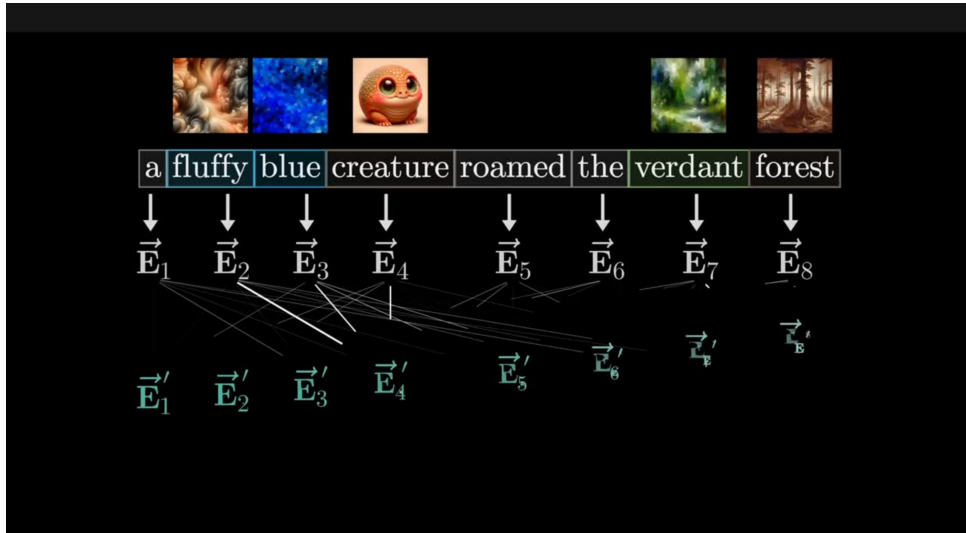
Definition 3.1 (Attention Head [Vaswani et al., 2017]). Let $X \in \mathbb{R}^{n \times d_{\text{in}}}$ be an input sequence of n vectors of dimension d_{in} . An attention head has learnable projection matrices

$$W_Q \in \mathbb{R}^{d_{\text{in}} \times d_k}, \quad W_K \in \mathbb{R}^{d_{\text{in}} \times d_k}, \quad W_V \in \mathbb{R}^{d_{\text{in}} \times d_v}.$$

The queries, keys, and values are $Q = XW_Q$, $K = XW_K$, $V = XW_V$, and the head output is

$$\text{AttnHead}(X) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Here softmax is applied row-wise so that each row of the attention-score matrix $QK^T/\sqrt{d_k}$ becomes a probability distribution summing to one. An attention *layer* concatenates the outputs of one or more such heads and projects the result.



Source: 3Blue1Brown on Youtube, "Attention in transformers, step-by-step | Deep Learning Chapter 6"

Figure 5: Visualisation of attention across token embeddings in the sentence “a fluffy blue creature roamed the verdant forest”. Each token’s representation $\vec{E}_i$ is updated to $\vec{E}'_i$ by attending to all other tokens, enabling context-sensitive meaning. Source: 3Blue1Brown [2024].

3.2 Permutation Equivariance

A key structural property of attention is that it is *permutation equivariant*: reordering the input sequence reorders the output in the same way. Formally, for any permutation matrix $P \in \mathbb{R}^{n \times n}$:

$$\text{Att}(PX) = P \text{Att}(X).$$

The proof proceeds in six steps. Permuting the input X by P yields $X' = PX$, which simultaneously permutes the queries, keys, and values: $Q' = PQ$, $K' = PK$, $V' = PV$. Consequently the attention-score matrix becomes $Q'K'^T = (PQ)(PK)^T = PQK^T P^T$.

Row-wise softmax commutes with symmetric permutations: $\text{softmax}(PAP^T) = P \text{softmax}(A) P^T$ for any matrix A , because P merely reorders rows and columns without mixing their entries. Using $A = QK^T/\sqrt{d_k}$ and combining with $V' = PV$ gives

$$\text{Att}(PX) = P \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \underbrace{P^T P}_=I V = P \text{Att}(X).$$

3.3 When to Break Equivariance

Permutation equivariance is desirable when the order of inputs is irrelevant (e.g. drug discovery, shopping basket recommendation, fraud detection). It is undesirable when order carries information, as in natural language (word order changes meaning), vision (spatial position matters), or autonomous driving (temporal ordering is critical). Positional encodings are the standard mechanism for breaking equivariance when needed.

3.4 Computational Complexity

Computing the $n \times n$ attention-score matrix QK^T requires $O(n^2)$ time and memory. This quadratic scaling is a significant bottleneck for long sequences and motivates architectures such as linear attention, sparse attention, and Sumformers, which aim to reduce this cost.

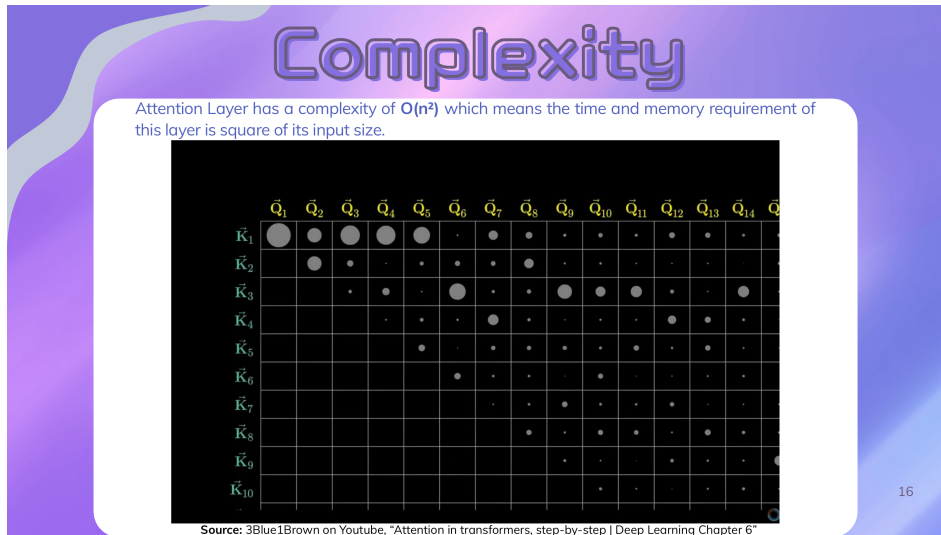


Figure 6: The $n \times n$ attention-score matrix QK^T between query vectors $\vec{Q}_1, \dots$ and key vectors $\vec{K}_1, \dots$. Each cell contains a dot-product score; circle size indicates magnitude. The full matrix must be computed, giving $O(n^2)$ complexity. Source: [3Blue1Brown \[2024\]](#).

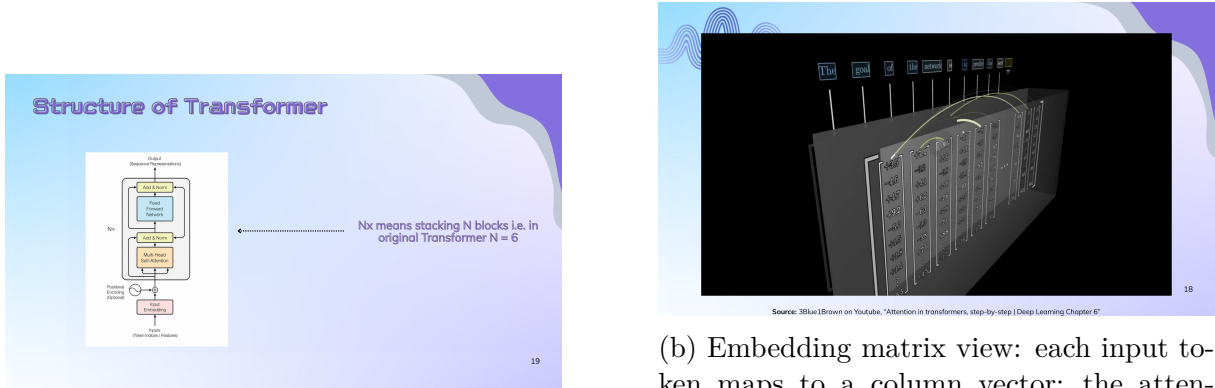
4 Transformers

4.1 Architecture

A Transformer encoder block consists of two sub-layers applied repeatedly N times (Figure 7a):

1. A **multi-head self-attention** layer, followed by residual connection and layer normalisation (Add & Norm).
2. A position-wise **feed-forward network** (FFN), again followed by Add & Norm.

The original architecture [Vaswani et al., 2017] uses $N = 6$ such blocks, each with 8 attention heads and model dimension $d_{\text{model}} = 512$. Optional **positional encodings** are added to the input embeddings before the first block; they inject information about token position, thereby breaking permutation equivariance.



(a) Transformer block diagram. The $N \times$ annotation indicates that the block is stacked N times ($N = 6$ in the original paper).

(b) Embedding matrix view: each input token maps to a column vector; the attention operation updates columns by attending across all others. Source: 3Blue1Brown [2024].

Figure 7: Transformer architecture and the embedding-matrix interpretation.

4.2 Permutation Equivariance of Transformers

Without positional encodings, every sub-layer of a Transformer block preserves permutation equivariance:

- Attention is permutation equivariant (proved in Section 3).
- Add & Norm applies element-wise operations per token, so it commutes with any permutation.
- The feed-forward network is applied independently to each token (the same parameters are shared across positions), so it too commutes with permutations.

Because the composition of permutation-equivariant continuous maps is again permutation-equivariant and continuous, a Transformer without positional encodings implements a **permutation-equivariant continuous sequence-to-sequence function**. This observation motivates the question addressed by Sumformers in the next section: *can every such function be approximated by a Transformer?*

5 Sumformers and Universal Approximation

5.1 Motivation

The UAT tells us that single-layer networks are universal approximators for scalar functions. An analogous question for Transformers is: can they approximate all continuous permutation-equivariant sequence-to-sequence functions? Alberti et al. [2023] answered this affirmatively by introducing a simplified architecture called a **Sumformer**, which is both theoretically tractable and practically interpretable.

5.2 Definition and Theorem

Definition 5.1 (Sumformer [Alberti et al., 2023]). Let $d' \in \mathbb{N}$ and let $\phi : \mathcal{X} \rightarrow \mathbb{R}^{d'}$ and $\psi : \mathcal{X} \times \mathbb{R}^{d'} \rightarrow \mathcal{Y}$ be two functions. A Sumformer $S : \mathcal{X}^n \rightarrow \mathcal{Y}^n$ is defined by first

computing a global context vector

$$\Sigma := \sum_{k=1}^n \phi(x_k),$$

and then applying a shared decoding function to each element together with the global context:

$$S([x_1, \dots, x_n]) := [\psi(x_1, \Sigma), \dots, \psi(x_n, \Sigma)].$$

Unlike full attention, which computes an $O(n^2)$ interaction matrix, the Sumformer aggregates global information in a single sum $\Sigma \in \mathbb{R}^{d'}$, making inference $O(n)$ in both time and memory.

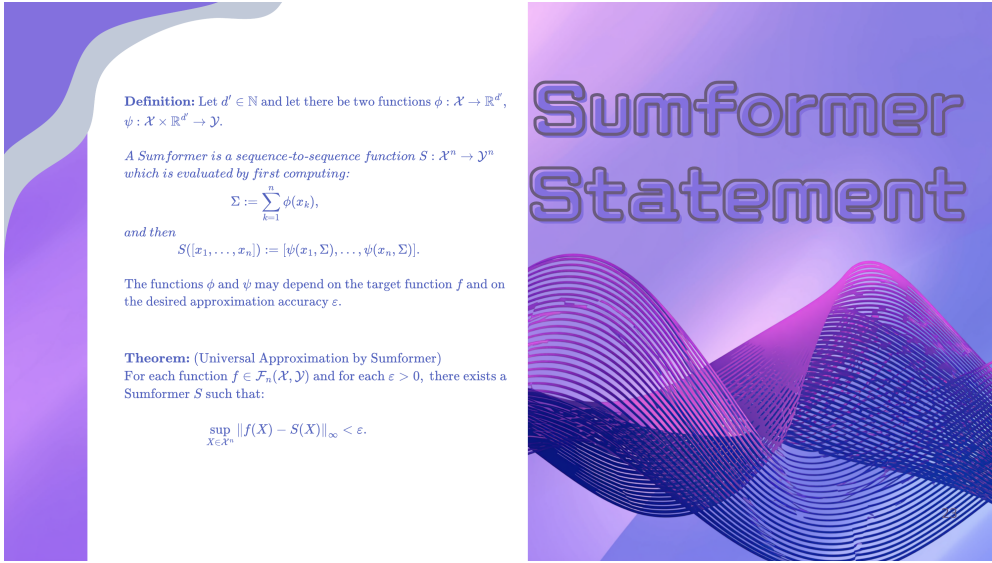


Figure 8: The Sumformer definition and the universal approximation theorem for continuous permutation-equivariant functions. The theorem guarantees that for any $f \in \mathcal{F}_n(\mathcal{X}, \mathcal{Y})$ and $\varepsilon > 0$ there exists a Sumformer S with $\sup_{X \in \mathcal{X}^n} \|f(X) - S(X)\|_\infty < \varepsilon$.

Theorem 5.2 (Universal Approximation by Sumformer [Alberti et al., 2023]). For each continuous permutation-equivariant function $f \in \mathcal{F}_n(\mathcal{X}, \mathcal{Y})$ defined on a compact set, and for each $\varepsilon > 0$, there exists a Sumformer S such that

$$\sup_{X \in \mathcal{X}^n} \|f(X) - S(X)\|_\infty < \varepsilon.$$

The proof leverages the classical UAT: because ϕ and ψ are themselves continuous functions on compact sets, they can each be approximated by a neural network. The sum aggregation is sufficient because any permutation-equivariant continuous function on $\mathcal{X}^n$ can be written in the form required by the Sumformer definition (this is established via a representation theorem for symmetric functions).

5.3 Sumformers in Practice: Where They Fall Short

Although Sumformers are universal approximators in theory, compressing all information into a single fixed-dimensional vector Σ is a bottleneck in practice. Chen et al. [2025] consider the task of estimating the radius of the *minimum enclosing ball* (MEB) of a

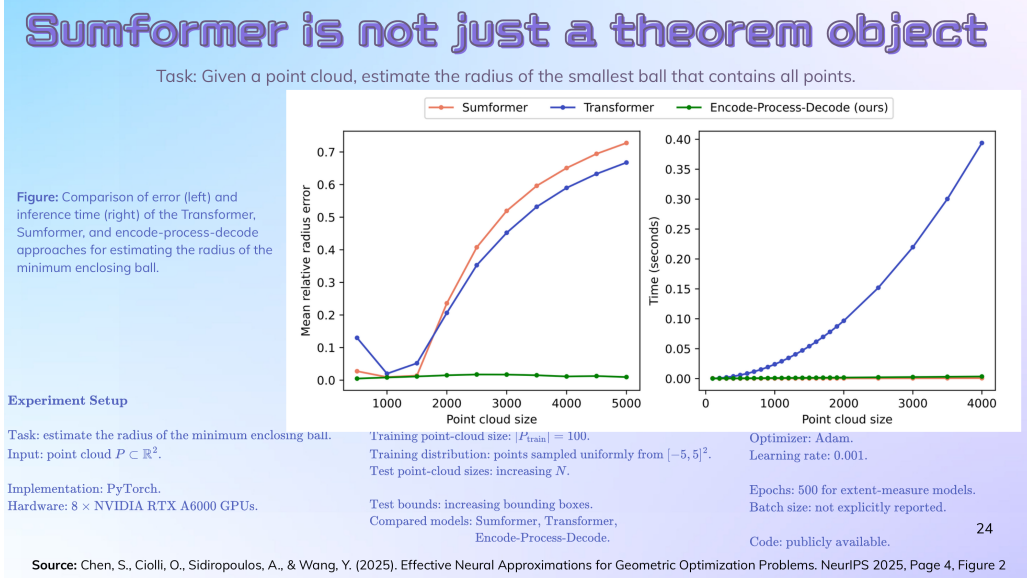


Figure 9: Mean relative radius error (left) and inference time (right) for estimating the MEB radius as a function of point-cloud size. The encode-process-decode model (green, *ours* in the original paper) achieves near-zero error at all sizes, while Sumformer (orange) and full Transformer (blue) diverge. The Transformer’s $O(n^2)$ cost is visible as a rapidly growing inference time. Source: [Chen et al. \[2025\]](#), Figure 2.

2D point cloud—a geometric problem where every point potentially contributes to the answer.

As Figure 9 shows, both the Sumformer and the standard Transformer fail to scale gracefully: the Sumformer’s error grows rapidly beyond $n \approx 1000$ because the single sum vector cannot capture the fine-grained geometric information needed to locate the farthest point. The Transformer is more accurate but pays a steep $O(n^2)$ inference-time penalty. The encode-process-decode approach of [Chen et al. \[2025\]](#) overcomes both limitations and suggests that architecture design beyond pure universal approximation arguments remains important.

6 Conclusion

This seminar traced the concept of universal approximation from single-hidden-layer feed-forward networks to sequence-to-sequence architectures. The UAT guarantees that wide shallow networks can approximate any continuous function, yet depth provides practical efficiency gains that width alone cannot replicate. The attention mechanism, at the core of Transformers, is intrinsically permutation-equivariant—a property that is useful for order-invariant tasks but must be broken with positional encodings for order-sensitive ones. Transformers without positional encodings are therefore permutation-equivariant continuous functions, and the Sumformer result by [Alberti et al. \[2023\]](#) shows that even a structurally much simpler architecture can approximate all such functions. However, the gap between theoretical expressiveness and practical performance, demonstrated by the MEB experiment, reminds us that universal approximation is a necessary but not sufficient condition for good generalisation at scale.

References

- Alberti, G., Clopath, C., and Voigtlaender, F. (2023). Sumformer: Universal Approximation for Efficient Transformers. *arXiv preprint arXiv:2307.02301*.
- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *ICLR 2015*.
- Chen, S., Ciolli, O., Sidiropoulos, A., and Wang, Y. (2025). Effective neural approximations for geometric optimization problems. In *NeurIPS 2025*.
- Chris, M. (2024). An elementary proof of a universal approximation theorem. *arXiv preprint*.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314.
- Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366.
- Nielsen, M. A. (2015). Neural networks and deep learning, Chapter 4. <http://neuralnetworksanddeeplearning.com/chap4.html>.
- Safran, I. and Shamir, O. (2020). Depth-width tradeoffs in approximating natural functions with neural networks. In *ICML 2017 (extended version)*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- 3Blue1Brown (2024). Attention in transformers, step-by-step | Deep Learning Chapter 6. YouTube. <https://www.youtube.com/watch?v=eMlx5fFNoYc>.